Cours 5

Vérification déductive

Logique de Hoare

Annotations JML : conception par contrats pour Java

Logique du premier ordre

Java et la logique dynamique

C/ACSL et Frama-C, avec N. Kosmatov (2 cours)

Preuve de programmes Java avec boucles et tableaux

Génération d'obligations de preuves, de Java/JML à JavaDL

Vérification incrémentale

Preuve interactive

Cours 5

Java et la logique dynamique

- JavaDL: Java Dynamic Logic
  - Logique interne de KeY
  - Logique de programmes adaptée à Java
  - KeY transforme chaque fichier Java/JML en conditions de correction en JavaDL (*voir prochain cours*)
- Dans ce cours
  - Découverte de JavaDL
  - Lien avec la logique de Hoare
  - Règles de déduction en logique dynamique
- Complément : "Reasoning about Programs with Dynamic Logic", Wolfgang Ahrendt, 14 October 2017

  http://www.cse.chalmers.se/edu/year/2017/course/TDA294_Formal_Methods_for_Software_Development/Lectures/Files/DynamicLogicCalculusPS.pdf

Ce document est protégé par des droits d'auteur et ne peut être traité, reproduit ou partagé par des systèmes automatisés sans autorisation explicite de ses auteurs.

Logique dynamique

Définitions

- La logique dynamique étend la logique du premier ordre avec les formules atomiques $\langle p \rangle \varphi$ et $[p]\varphi$, où p est un programme et φ est une formule quelconque.
- $\langle p \rangle \varphi$ s'appelle la plus faible précondition sous laquelle p établit φ ($wp(p, \varphi)$, *weakest precondition*)
  - Précondition : Dans tout état tel que $\langle p \rangle \varphi$ est vraie avant exécution du programme p, p **termine et** φ est vraie après exécution de p.
  - La plus faible : Si ψ est une autre telle précondition, alors ψ implique $\langle p \rangle \varphi$.
- $[p]\varphi$ s'appelle la plus faible précondition libérale sous laquelle p établit φ ($wlp(p, \varphi)$, *weakest liberal precondition*)
  - Précondition **libérale** : Dans tout état tel que $[p]\varphi$ est vraie avant exécution du programme p, **si** p **termine alors** φ est vraie après exécution de p.
  - La plus faible : Tout autre telle précondition implique $[p]\varphi$

Logique dynamique

Lien avec la logique de Hoare

- Correspondance entre logique dynamique et logique de Hoare

$$\psi \Rightarrow [p]\varphi$$
si et seulement si
$$\{\psi\}p\{\varphi\}$$

- $\{\psi\}p\{\varphi\}$ est un triplet de Hoare
- Attention : Bien préciser la nature des programmes (p) et des formules (φ, ψ)

Théorème fondamental de la correction des programmes

Correction totale = correction partielle + terminaison

$$wp = wlp + termination$$

- Correction partielle : correction sous hypothèse de terminaison
- Décompose le problème de la correction en deux sous-problèmes très différents ("*diviser pour régner*")
- Dans cette partie du cours, on n'étudie que la correction partielle, pas la terminaison

JavaDL: Java Dynamic Logic

Logique dynamique pour les programmes Java

JavaDL

- Un *programme* JavaDL est défini par la grammaire suivante, où ψ est une formule logique du premier ordre :

$$p, q \ ::= \ var \ \texttt{=} \ expr \texttt{;} \mid \texttt{\{} \ p \ q \ \texttt{\}} \mid \texttt{if} \ (\psi) \ p \mid \texttt{if} \ (\psi) \ p \ \texttt{else} \ q \mid \texttt{while} \ (\psi) \ p$$

- Une *formule atomique* JavaDL est soit une formule atomique du premier ordre, soit de la forme $\langle p \rangle \varphi$, soit de la forme $[p]\varphi$, pour un programme JavaDL p et une formule JavaDL φ quelconques
- Une *formule* JavaDL est une combinaison finie quelconque de formules JavaDL atomiques, par des connecteurs logiques (`&`, `|`, `!`, `->`) et des quantifications (`\forall`, `\exists`)
- Exemple : `k = 0 -> \[ k = k+1; \] k = 1`

JavaDL

Formules JavaDL

- Grammaire (simplifiée)

$$\begin{array}{lcl} \varphi & ::= & \texttt{!}\ \varphi \mid \varphi \ op \ \varphi \mid \texttt{\backslash forall}\ T\ v;\ (\varphi) \mid \texttt{\backslash exists}\ T\ v;\ (\varphi) \mid \\ & & \langle p \rangle\, \varphi \mid [p]\, \varphi \mid expr \ rel \ expr \\ op & ::= & \texttt{\&} \mid \texttt{|} \mid \texttt{->} \mid \ldots \\ rel & ::= & \texttt{=} \mid \texttt{>} \mid \texttt{>=} \mid \texttt{<} \mid \texttt{<=} \mid \ldots \end{array}$$

- Attention : Ne pas confondre les syntaxes JavaDL et JML !
- En JavaDL, que des `\forall` et `\exists` en deux parties
- $\langle \ldots \rangle$ est codé par `\<...\>`
- $[\ldots]$ est codé par `\[...\]`

Exemple 1

Échange de deux variables entières

```
\programVariables {
 int x;
 int y;
}

\problem {
 \forall int x0;
  \forall int y0;
   ((x = x0 & y = y0) ->
    \[ {x = x+y; y = x-y; x = x-y;} \] (x = y0 & y = x0))
}
```

Démonstration avec le prouveur KeY ...

Exemple 2

Initialisation d'un tableau de taille 10

```
\programVariables {
 int k;
 int[] tab;
}
\problem {
 tab != null & tab.length = 10 & wellFormed(heap) ->
 \[
   {
    k = 0;
    while (k < 10) tab[k++] = 3;
   }
 \] (\forall int j; (0 <= j & j < 10 -> tab[j] = 3))
}
```

Démonstration avec le prouveur KeY ...

Rapport de preuve : .....
..... [R_{25}]

Exemple 3

Initialisation d'un tableau de taille inconnue

```
\programVariables {
 int k;
 int[] tab;
}
\problem {
 (tab != null & tab.length >= 0 & wellFormed(heap)) ->
 \[
   {
    k = 0;
    while (k < tab.length) tab[k++] = 3;
   }
 \] (\forall int j; (0 <= j & j < tab.length -> tab[j] = 3))
}
```

Rapport de preuve : .....
.....
.....
..... [R_{26}]

Logique dynamique avec substitutions explicites

Extension du calcul des séquents à JavaDL

- Un *séquent* est de la forme

$$H_1, \ldots, H_m \Longrightarrow C_1, \ldots, C_n$$

  où H_1, ..., H_m, C_1, ..., C_n sont des formules
- Dans cette logique, les formules sont de la forme $\{\mathcal{U}\}\ F$
  - $\mathcal{U}$ est une séquence de substitutions explicites
  - F est une formule JavaDL
- Une *substitution explicite* des variables x_1, ..., x_n (toutes distinctes) par les expressions e_1, ..., e_n est notée $x_1 := e_1 \,||\, \ldots \,||\, x_n := e_n$
- L'expression $\{s_1, \ldots, s_k\}\ e$ représente l'application des substitutions explicites s_k, puis s_{k-1}, ..., puis s_1 à l'expression e

Logique dynamique avec substitutions explicites

- Règle de déduction pour l'affectation

$$\frac{C \Longrightarrow \{\mathcal{U}, x := e\}\ F, D}{C \Longrightarrow \{\mathcal{U}\}\ [x = e;]\ F, D} \qquad (\textit{affectation})$$

Logique dynamique avec substitutions explicites
Bloc d'instructions

$$\frac{C \Longrightarrow \{\mathcal{U}\}\ [p]\,[q]\,F, D}{C \Longrightarrow \{\mathcal{U}\}\ [\{p\ q\}]\,F, D} \qquad (bloc)$$

- $\{\ p_1\ \ldots p_n\ \}$ abrège $\{\ p_1\ \{\ \ldots\ \{\ p_{n-1}\ p_n\ \}\ \ldots\ \}$
- La règle (*bloc*) n'existe pas dans le prouveur KeY, mais elle permet de diminuer le nombre de règles

Logique dynamique avec substitutions explicites
Instructions conditionnelles

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]\,F), D \\ C \Longrightarrow \{\mathcal{U}\}\ (\texttt{(!}\ b\texttt{)}\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{if}\ \ \texttt{(}b\texttt{)}\ \ p]\,F, D} \qquad (si\text{-}alors)$$

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]F), D \\ C \Longrightarrow \{\mathcal{U}\}\ (\texttt{(!}b\texttt{)}\ \texttt{->}\ [q]\,F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}[\texttt{if}\ \ \texttt{(}b\texttt{)}\ \ p\ \texttt{else}\ \ q]\,F, D} \qquad (si\text{-}alors\text{-}sinon)$$

Logique dynamique avec substitutions explicites
Itération

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ I, D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ (\texttt{(}I\ \&\ b\texttt{)}\ \texttt{->}\ [p]\,I), D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ (\texttt{(}I\ \&\ !\ b\texttt{)}\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{while}\ \ \texttt{(}b\texttt{)}\ \ p]\,F, D} \qquad (tant\text{-}que)$$

- Fournir un invariant de boucle I
- La substitution $\mathcal{V}$ rend anonymes les variables que le programme p peut modifier
  - Clause `assignable` en JML
  - Paramètre Modifiable de la règle Loop Invariant

Règle Loop Invariant dans KeY (1/2)
sur l'exemple 3

1. Charger le fichier `.key`

```
==>
    !tab = null & tab.length >= 0 & wellFormed(heap)
 -> \[{
      k = 0;
      while (k < tab.length) tab[k++] = 3;
    }\] \forall int j; (0 <= j & j < tab.length -> tab[j] = 3)
```

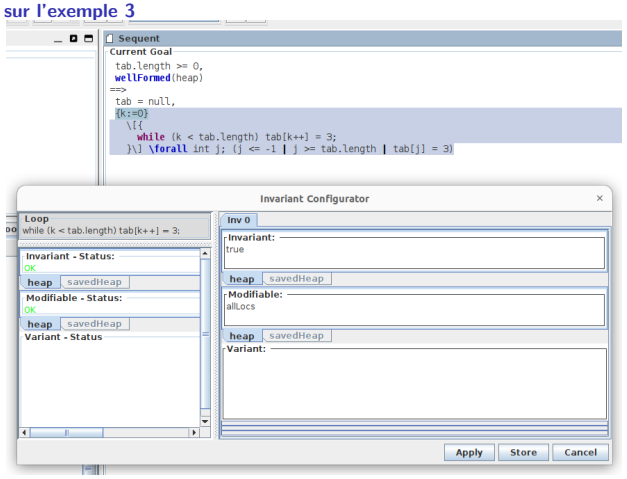
2. Cliquer sur pour lancer la preuve automatique

```
 tab.length >= 0,
 wellFormed(heap)
==>
 tab = null,
 {k:=0}
   \[{
     while (k < tab.length) tab[k++] = 3;
   }\] \forall int j; (j <= -1 | j >= tab.length | tab[j] = 3)
```

3. Sélectionner la formule `{..} \[{ .. }\] ..` et choisir Loop Invariant dans le menu déroulant

Règle Loop Invariant dans KeY (2/2)

sur l'exemple 3



- Attention : avec KeY 2.12.3, ne pas modifier le champ Modifiable:
- Toute modification de la valeur de ce champ désactive les boutons Apply et Store
- La valeur par défaut (allLocs) de ce champ convient

Application des substitutions

Explicit state update

- Lorsqu'aucune de ces règles ne s'applique plus, le séquent est de la forme
$$C \Longrightarrow \{\mathcal{U}\}\ F, D$$
où le symbole principal de F est un connecteur logique ou un quantificateur
- On réduit alors l'expression $\{\mathcal{U}\}\ F$ par application de $\mathcal{U}$ comme une substitution
- Pour réduire l'expression $\{\mathcal{U}, s\}\ F$, on commence par appliquer s à F, puis on applique $\mathcal{U}$ au résultat
- Voir "Reasoning about Programs with Dynamic Logic" pages 9-11 et 13-17 pour l'application des *sequential updates* et *parallel updates*

Logique dynamique avec substitutions explicites

Règles de déduction (1/2)

$$\frac{C \Longrightarrow \{\mathcal{U}, x := e\}\ F, D}{C \Longrightarrow \{\mathcal{U}\}\ [x = e;]\ F, D} \quad (\textit{affectation})$$

$$\frac{C \Longrightarrow \{\mathcal{U}\}\ [p]\,[q]\ F, D}{C \Longrightarrow \{\mathcal{U}\}\ [\{p\ q\}]\ F, D} \quad (\textit{bloc})$$

où $\{\ p_1\ \ldots p_n\ \}$ abrège $\{\ p_1\ \{\ \ldots\ \{\ p_{n-1}\ p_n\ \}\ \ldots\ \}$

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]\ F), D \\ C \Longrightarrow \{\mathcal{U}\}\ ((!\ b)\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{if}\ \ (b)\ p]\ F, D} \quad (\textit{si-alors})$$

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]F), D \\ C \Longrightarrow \{\mathcal{U}\}\ ((!b)\ \texttt{->}\ [q]\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}[\texttt{if}\ \ (b)\ \ p\ \texttt{else}\ \ q]\ F, D} \quad (\textit{si-alors-sinon})$$

Logique dynamique avec substitutions explicites

Règles de déduction (2/2)

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ I, D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ ((I\ \&\ b)\ \texttt{->}\ [p]\ I), D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ ((I\ \&\ !\ b)\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{while}\ \ (b)\ \ p]\ F, D} \quad (\textit{tant-que})$$

où la substitution $\mathcal{V}$ rafraîchit (remplace par des variables fraiches) toutes les variables que le programme p peut modifier

- Trouver l'invariant de boucle I

Exemple d'application de la règle (tant-que)

$$\frac{\begin{array}{c} i = 0 \Longrightarrow \{k := 0\}\ I \\ i = 0 \Longrightarrow \{k := 0, i := i_1, k := k_1\}\ ((I\ \&\ k < n)\ \texttt{->}\ [p]\, I) \\ i = 0 \Longrightarrow \{k := 0, i := i_1, k := k_1\}\ ((I\ \&\ !\ k < n)\ \texttt{->}\ (i = n)) \end{array}}{i = 0 \Longrightarrow \{k := 0\}[\texttt{while}\ (k < n)\ \{i = i + 1; k = k + 1;\}](i = n)}$$

- Invariant de boucle I : $k = i\ \&\ k \leq n$
- Programme p : $\{i = i + 1; k = k + 1;\}$
- Variables modifiées : i et k ($\mathcal{V}$ est $i := i_1, k := k_1$)

Réduction de la 3e condition

$$i = 0 \Longrightarrow \{k := 0, i := i_1, k := k_1\}\ ((k = i\ \&\ k \leq n\ \&\ !\ k < n)\ \texttt{->}\ (i = n))$$

$\rightarrow$ ..... $[R_{27}]$

$\rightarrow$ ..... $[R_{28}]$

$\rightarrow$ ..... $[R_{29}]$

Par raisonnement arithmétique, la conclusion devient

..... $[R_{30}]$

ce qui permet de conclure, par transitivité et symétrie de l'égalité